

Anteater Poker Software Specification

Version 0.1 Alpha Draft

By Jim Truong, Hamza Vaid, Shogo Stuck, Ben Choi,
Giovanna Dunker Estruquel
University of California, Irvine

Table of Contents

Glossary.....	3
1. Poker Client Software Architecture Overview.....	6
1.1 Main data types and structures.....	6
1.2. Major software components.....	7
1.3. Module interfaces.....	8
1.4. Overall program control flow.....	9
1.5. Automated Client: Poker Bot.....	9
2. Poker Server Software Architecture Overview.....	10
2.1. Main data types and structures.....	10
2.2. Major software components.....	11
2.3. Module interfaces.....	12
2.4. Overall program control flow.....	13
3. Installation.....	14
3.1. System requirements, compatibility.....	14
3.2. Unpacking and configuration.....	14
3.3. Building, compilation, installation.....	15
4. Documentation of packages, modules, interfaces.....	16
4.1. Detailed description of data structures.....	16
4.2. Detailed description of functions and parameters.....	18
4.3. Detailed description of the communication protocol.....	20
5. Development plan and timeline.....	21
5.1. Partitioning of tasks.....	21
5.2. Team member responsibilities.....	21
Back Matter.....	21
Copyright.....	21
References.....	22
Index.....	23

Glossary

- **AbilityCard** – A data structure that represents an Anteater ability card. It stores the ability type, whether it has been used, the owner seat, target seat, and optional parameters.
- **ActionType** – An enum used to represent player actions such as CHECK, CALL, RAISE, FOLD, USE_ABILITY, or LEAVE.
- **All-In** – A player bets all their remaining chips on a single hand. They cannot be forced out of the pot because they run out of chips; instead, a side pot may be created.
- **Ante** – A small, mandatory bet required from all players before a hand begins to seed the pot.
- **Anteater Ability Card** – A special card used in Anteater Poker that gives a player a unique ability during gameplay.
- **Big Blind** – A mandatory bet posted by the player two seats to the left of the button. It is usually double the size of the small blind.
- **Bet** – The act of placing an initial wager during a betting round.
- **Bluff** – To bet or raise with a hand that is unlikely to be the best, with the goal of making opponents fold better hands.
- **Board / Community Cards** – The five cards dealt face-up in the center of the table that all players use with their private cards to form the best hand.
- **Bot Player** – An automated poker client that connects through the same socket protocol as a human player and submits legal player actions.
- **Broadcast** – A server operation that sends the current game state to all connected clients.
- **Burn Card** – The top card of the deck that is set aside face-down before dealing community cards.
- **Call** – To match the current highest bet or raise in the betting round.
- **Card** – A data structure that represents a playing card using rank, suit, and visibility fields.
- **Check** – To pass the action to the next player without betting, allowed only if no one has made a bet in the current round.
- **Client** – The player-side program that handles the GUI, user input, local display, and communication with the server.
- **ClientState** – A client-side data structure that stores connection status, player name, seat, visible table state, private hand, ability card, and GUI button flags.

- **Community Cards** – Shared face-up cards used by all players to make the best five-card hand. These include the flop, turn, and river.
- **Configuration Options** – Command-line settings such as host, port, name, bots, starting points, and log file location.
- **Deck** – A data structure that stores the 52 cards used in the game, along with the current top card index and remaining card count.
- **Flop** – The first three community cards dealt after the first round of betting.
- **Fold** – To discard a hand and give up interest in the current pot.
- **GameState** – A server-side data structure that stores the official game state, including current turn, players, pot, public cards, and phase information.
- **GTK** – The graphical user interface library used to build the client and server GUI windows.
- **Hand Ranking** – The order used to determine the strength of poker hands, such as royal flush, straight flush, full house, pair, or high card.
- **Hole Cards** – The private cards dealt face-down to each player.
- **Main Pot** – The main pool of chips that all active players are eligible to win.
- **Module Interface** – The set of functions that allows different parts of the program to communicate with each other.
- **Muck** – To fold a hand without showing the cards; also refers to the pile of discarded cards.
- **NetworkMessage** – A serialized message container used for client-server communication. It stores message type, payload, payload size, and sender or seat information.
- **Player** – A server-side data structure that stores a player's name, chips, socket, hand, and current status.
- **PlayerAction** – A data structure representing a move chosen by the player, such as check, call, raise, fold, use ability, or leave.
- **PlayerView** – A client-side view of one player seat, including display name, seat index, point total, folded/active state, and current bet.
- **Pot** – The total number of chips wagered by players during a hand.
- **PrivateHand** – A data structure that stores the player's private cards and assigned Anteater ability card.
- **Raise** – To increase the current bet, requiring other players to call, fold, or raise again.
- **River** – The fifth and final community card dealt.

- **RuleResult** – A rules-module return value that stores whether an action is valid, the reason, winner information, and hand rank.
 - **Server** – The authoritative program that controls the official game state, validates moves, handles betting, deals cards, and broadcasts updates.
 - **Showdown** – The final phase of a hand where remaining players reveal cards and the winner is determined.
 - **Side Pot** – A separate pot created when one player goes all-in and other players continue betting.
 - **Small Blind** – A mandatory bet posted by the player immediately to the left of the button.
 - **Socket** – A network communication endpoint used by the client and server to exchange messages.
 - **TCP/IP** – The network protocol used for client-server communication in the poker game.
 - **Turn** – The fourth community card dealt face-up.
-

Hand Rankings, Highest to Lowest

- **Royal Flush** – A, K, Q, J, 10 of the same suit.
- **Straight Flush** – Five consecutive cards of the same suit.
- **Four of a Kind / Quads** – Four cards of the same rank.
- **Full House / Boat** – Three of a kind combined with a pair.
- **Flush** – Five cards of the same suit, not in sequence.
- **Straight** – Five consecutive cards of different suits.
- **Three of a Kind / Trips / Set** – Three cards of the same rank.
- **Two Pair** – Two different pairs in one hand.
- **One Pair** – Two cards of the same rank.
- **High Card** – The highest card in the hand when no other ranking is met.

1. Poker Client Software Architecture Overview

1.1 Main data types and structures

The client is responsible for user interaction, local display, input collection, and network communication. It does not decide whether an action is legal. The client sends requests to the server and updates the display only after receiving confirmed game state messages from the server.

Data Type	File	Purpose
ClientState	client_state.h	Stores local connection status, logged-in player name, number of logged-in players, assigned seat, visible public table state, private hand, ability card, and flags that control which GUI buttons are enabled
PlayerView	client_state.h	Client-side view of one player seat: display name, seat index, point total, folded/active state, and current bet. Private cards for other players are never stored here.
PlayerAction	poker_rules.h	Represents an action chosen by the player, such as check, call, raise, fold, use ability, or leave. It stores the action type and any needed amount, such as the raise value.
Card	cards.h	Represents a normal playing card using suit, rank, and visibility fields. Used by both client display and server rules.
PrivateHand	cards.h	Stores the player's private cards. In Anteater Poker, this includes the two normal poker cards and the assigned Anteater ability card.
AbilityCard	anteater_abilities.h	Represents one Anteater ability card, its type, whether it has been used, target seat if needed, and any selected parameter.
NetworkMessage	socket_client.h	Serialized message container with type, payload size, payload string/fields, and sender/seat metadata.
GuiContext	client_gui.h	Holds GTK widgets for the table display, action buttons, login window, card images, status label, and callbacks.

Table 1: Data Structures featured in Anteater Poker and their location and purpose.

The client-side data structures are designed to separate the visible game display from the official game logic. The client stores only the information needed to show the table, display the player's private hand, collect user input, and communicate with the server. It does not directly decide whether a move is legal or who wins the hand. Instead, the client sends a PlayerAction message to the server and waits for a confirmed GAME_STATE update before changing the visible table. This prevents modified clients from cheating or changing the game state locally. We introduce several enums to help us structure the information in an efficient, readable way. Including Suit, Rank, ActionType.

1.2. Major software components

The client side is divided into interface, networking, state management, poker display, ability display, and bot automation. This separation keeps the GUI responsive while the socket layer receives updates from the server.

Component	Responsibility
src/client/poker.c	Client program entry point. Parses command-line options, initializes GTK, opens network connection, and starts the client event loop.
src/client/socket_client.c	Connects to the server, sends login/action messages, receives updates, and converts raw socket data into NetworkMessage values.
src/client/client_state.c	Applies server-approved updates to the local ClientState and provides helper functions for GUI rendering.
src/gui/client_gui.c	Builds the client graphical table, card display, buttons, seat labels, and status messages.
src/bot/bot_player.c	Automated client logic for empty seats. It uses the same socket protocol as a human client.
assets/...	Card images, Anteater ability icons, and other GUI images used by the client display.

Client Module Hierarchy:

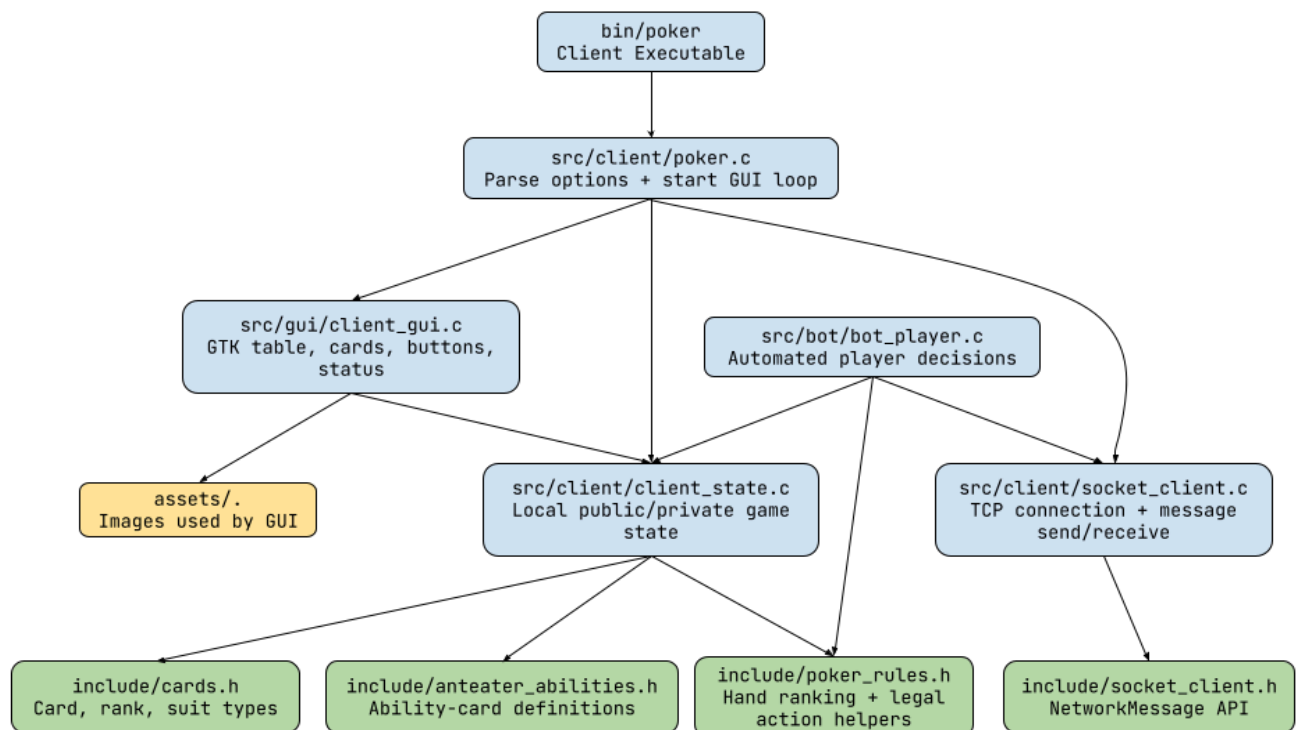


Figure 1 and Table 2: Client module hierarchy and major include dependencies

1.3. Module interfaces

The following API functions are the main interfaces used by the client modules. Function names and parameters may be adjusted during implementation, but this table defines the intended boundaries between modules.

Category	Function	Responsibility
GUI	<code>void init_gui()</code>	Responsible for starting the GTK window for the client and displaying the game information available to the player.
GUI	<code>void update_gui()</code>	Refreshes GTK upon new information, such as changes in ClientState. Also important when certain buttons in the GUI should be hidden or shown.
Game Flow	<code>void make_action()</code>	Creates a PlayerAction object and updates the ClientState, as well as notifying the server about the change.
Client	<code>int connect_to_server(char *ip, int port)</code>	Uses <code>socket()</code> and <code>connect()</code> to connect to our poker server. Takes an IP address and port number as arguments. Returns the corresponding file descriptor.
Client	<code>void msg_to_server(int socket_fd, ActionType action)</code>	Sends a message to the server notifying that a new action has occurred at the table. Takes the file descriptor and action as arguments.
Client	<code>int check_server_updates(int socket_fd, char *buf)</code>	Receive a message from the server using <code>recv()</code> , filling the buffer with the data. Takes the file descriptor and buffer pointer as arguments. Returns a number corresponding to successful or unsuccessful completion.

Table 3: API interface functions used by client.

1.4. Overall program control flow

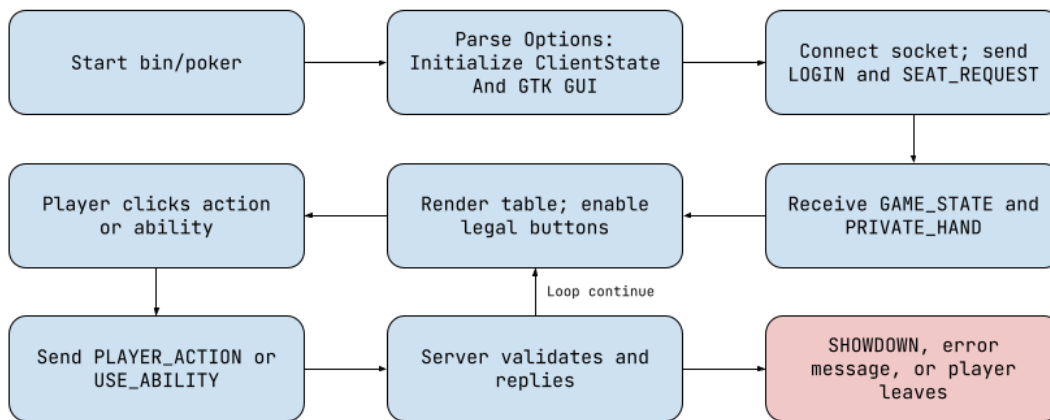


Figure 2: Client overall program control flow.

When the poker client starts, the program first reads command-line options such as the host, port, and optional player name. It then initializes the GTK graphical interface and attempts to connect to the poker server. After connecting, the client sends a login request with the player's display name. If the server accepts the connection, the client receives its assigned seat, starting points, private cards, and ability card. During the game, the client waits for server updates and refreshes the GUI whenever a new game state is received. When it is the player's turn, the GUI enables the correct buttons, such as check, call, raise, fold, or use ability. When the player clicks a button, the GUI creates a `PlayerAction` and sends it to the server. The client does not immediately change the official game state. Instead, it waits for the server to confirm the action and send back an updated table state. This control flow keeps the game fair because all important rules, turns, betting, and card changes are handled by the server.

1.5. Automated Client: Poker Bot

The poker bot will be designed as an automated client, not as server-side cheating logic. It connects through the same socket protocol, receives only the information a player is allowed to know, and submits normal player actions.

- Input: `GAME_STATE`, `PRIVATE_HAND`, `ACTION_REQUEST`, and legal action flags sent by the server.
- Output: one `PlayerAction` message at a time, such as `CHECK`, `CALL`, `RAISE`, `FOLD`, or `USE_ABILITY`.
- Decision Strategy: check when possible, call small bets with reasonable hand strength, fold weak hands to large raises, and use an Anteater ability when the simple rule table says the ability is beneficial.

2. Poker Server Software Architecture Overview

2.1. Main data types and structures

The data types and structures we need in the server will be very similar to the data types and structures used in the client. We need some object to handle the state of the game, we can call this GameState. We need an object to handle the player, we can just call this Player. Finally, we need an object to handle client requests and parse them into actionable instructions, we'll call this NetworkMessage.

We can also use the same enum as the client to help us organize information by introducing ActionType, with possible values including CHECK, CALL, RAISE, and FOLD.

Data Type	File	Purpose
GameState	game_state.h	Holds the network socket, current player turn, as well as the points in the pot and public cards (flop, turn, river). Accessible to every player's client through the server.
GamePhase	game_state.h	Enum that tracks the current phase of the hand, such as LOBBY, PREFLOP, FLOP, TURN, RIVER, SHOWDOWN, and GAME_OVER.
ServerConfig	game_state.h	Stores server startup options, including port number, table name, starting points, bot count, and log file path.
Player	game_state.h	Information about the current player as well as the socket, including the player's name, their chips, their hand, and their current status.
Scoreboard	game_state.h	Stores player scores across rounds and is used to rank players when the game session ends.
Card	cards.h	Normal playing card with rank, suit, and visibility. The server can store hidden cards; client receive only authorized cards.
Deck	deck.h	Array of Card values plus top index/count. Used for shuffling and dealing.
NetworkMessage	socket_server.h	Message envelope used to parse client commands and build server replies.
ClientConnection	socket_server.h	Stores socket file descriptor, seat number, connection status, and address information for each connected player.
RuleResult	poker_rules.h	Return value for rule validation and hand evaluation, including validity flag, reason string, winner, and hand rank.

Table 4: Server-side data types and structures.

2.2. Major software components

Component	Responsibility
src/server/server.c	Server program entry point. Creates the remote server and begins listening for server requests from the client application.
src/server/game_state.c	The server-side module that keeps track of the game elements, such as the players, pot, and cards.
src/server/socket_server.c	Receives messages from the client and handles them by parsing them into ClientCommands and then changing the GameState accordingly, then relaying that information to each connected client.
src/rules/cards.c	Card helper functions such as card creation, rank/suit conversion, comparison, and display strings.
src/rules/deck.c	Deck creation, shuffling, drawing, and validation.
src/rules/poker_rules.c	Texas Hold'em hand ranking, betting legality, winner determination, and point/pot update rules.
src/gui/server_gui.c	Server control-room GUI for table status, connected clients, logs, scoreboard, and host controls.
src/bot/bot_player.c	Optional automated player module used to occupy empty seats.

Table 5: Server-side module hierarchy and source organization.

Server Module Hierarchy:

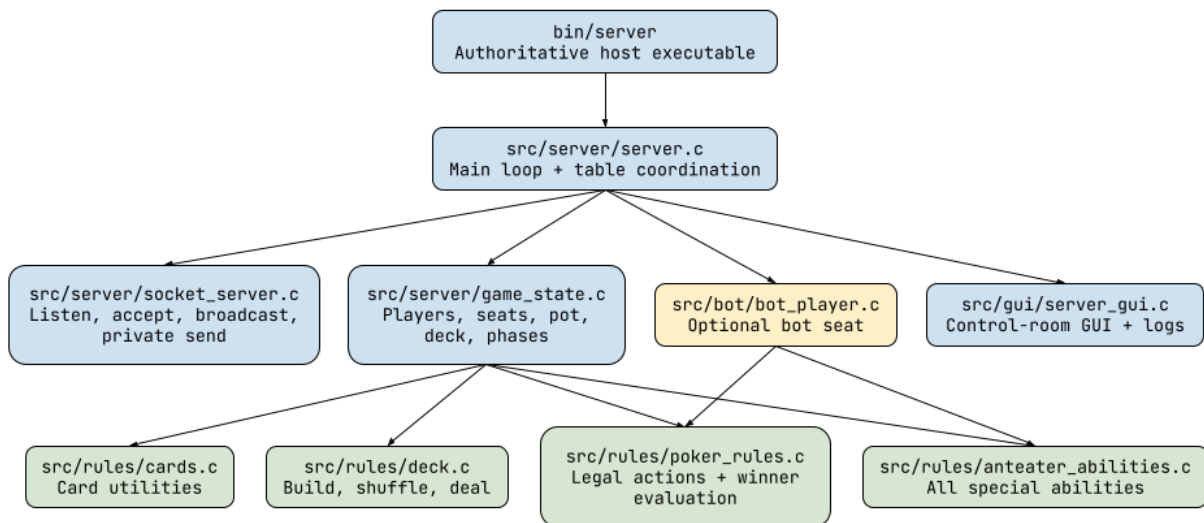


Figure 3: Server-side module graph

2.3. Module interfaces

The server side is concerned with parsing information from the client, but it also updates the game so that it can send back updates to each client as well. The following API functions are the main interfaces used by the server modules. Function names and parameters may be adjusted during implementation.

Function	Responsibility
<code>int init_server(int port)</code>	Responsible for starting the remote server and listening for server requests. It uses <code>bind()</code> to put a socket on the given port and returns the file descriptor.
<code>int accept_client(int sock)</code>	Adds a new client to the server when <code>select()</code> changes due to a new connection. It uses <code>accept()</code> to connect the two and then returns the socket descriptor.
<code>void broadcast_game(int fds[], int num_clients, char *msg)</code>	Broadcasts the current state of the game to all active clients. This function can be used after the player turn is incremented. The arguments are the file descriptors of the clients
<code>NetworkMessage parse_client_msg(char *buf)</code>	Parses the buffer with the client's message into a <code>NetworkMessage</code> object that can be processed by the server.
<code>Player calc_winner()</code>	Used after all but one player folds, or the end of the river is reached, to calculate who should win the pot. Returns the winning player.

Table 6: API interface functions used by server.

2.4. Overall program control flow

- Initialization: server is booted up and initializes the socket user on a designated port
- Lobby Phase: the client connects to the server. The connection manager assigns an ID and seats them in a virtual table
- Pregame: game engine shuffle the deck, take the points of small and big blind players, distribute two cards to each socket connection, change game phase to pre-flop
- Action Loop: server is in blocking/polling state
 - When a valid move is received → update pot and game chips
 - Send the new state to all clients
 - Turn the pointer advances
- Progression: Once all bets are matched in a round, the server deals community cards (Flop, Turn, River). A winner is determined by, and chips are awarded. Server reset table structure for the next hand

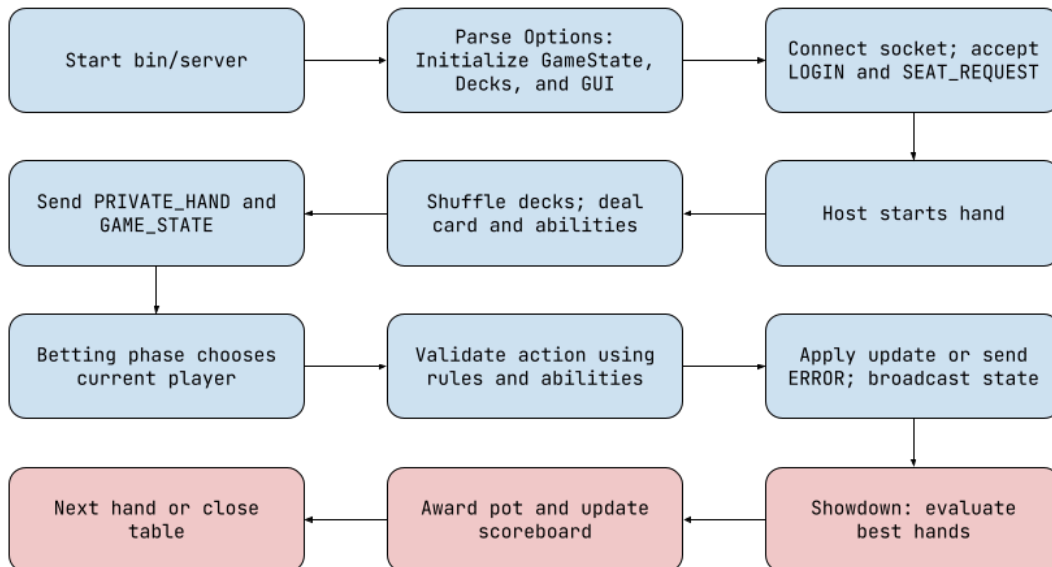


Figure 4: Server-side control flow

3. Installation

3.1. System requirements, compatibility

- Operating System: Windows / macOS / Linux
- Compiler: GCC (C language), make, gtar, and standard C libraries.
- GUI Libraries: GTK 3
- Memory: At least 512 MB recommended.
- Storage: At least 50 MB available space.

3.2. Unpacking and configuration

Currently, the program has not yet been developed, so a complete setup and configuration process is not available. However, the expected installation procedure is as follows:

1. Download and extract the compiled program files.
`gtar xvzf Poker_V1.0.tar.gz`
2. Open terminal and navigate to project directory.
`cd poker`
3. Before starting a game, the host should choose a port and confirm that other players can reach the server. The default port will be 10010. If the port is blocked or already in use, choose another port and share it with the rest of the players.

Example Server Start:

`./bin/server --port 10010 --table "ZotHouse"`

Example Client Start:

`./bin/poker --host server_name_or_ip --port 10010`

Configuration Options:

Option	Example	Purpose
--host	--host crystalcove.ics.uci.edu	Server name or IP address used by client.
--port	--port 10010	TCP port used by both servers and client.
--name	--name Hamza	Optional player display name. If not added, the login window will prompt the user.
--bots	--bots 1	Number of bot players that the server may add to empty seats.
--points	--points 1000	Starting points for each player.
--log	--log logs/game.log	Location of game log for debugging.

Table 7: Configuration Options for game start.

3.3. Building, compilation, installation

The Makefile should build both executables, bin/server and bin/poker. Object files may be placed in a temporary build directory or cleaned after compilation. Unit tests should be callable through make test and should include tests for cards, deck, poker rules, Anteater abilities, and one system test for client-server behavior.

Download and extract the compiled program files.

gtar xvzf Poker_V1.0_src.tar.gz

Open terminal and navigate to project directory.

cd poker

Compile the Source code.

make

make test

make clean

To run the code follow 3.2.

4. Documentation of packages, modules, interfaces

4.1. Detailed description of data structures

Card and Deck:

```
typedef enum {
    SUIT_HEARTS = 0, SUIT_DIAMONDS, SUIT_CLUBS, SUIT_SPADES,
} Suit;
```

The suit data structure represents the 4 standard suits in a deck of cards.

```
typedef enum {
    RANK_TWO = 2, RANK_THREE, RANK_FOUR, RANK_FIVE,
    RANK_SIX, RANK_SEVEN, RANK_EIGHT, RANK_NINE,
    RANK_TEN, RANK_JACK, RANK_QUEEN, RANK_KING,
    RANK_ACE, RANK_INVALID
} Rank;
```

The rank data structure represents the 13 standard ranks in a suit of cards. The only exception being that Joker is not included because Texas hold 'em poker does not use the Joker rank. All number constants are mapped to their face values. (e.g. RANK_TWO = 2, RANK_THREE = 3, etc.) JACK, QUEEN, KING, and ACE are represented as 11, 12, 13, and 14 respectively.

```
typedef struct {
    Rank rank;
    Suit suit;
    int visible;
} Card;
```

The card data structure encapsulates the 2 aspects of a card, the Rank and the Suit. This way, the program can treat each card as an uniform object with a given rank and suit value. In poker, 52 cards will be created and used. There is an added visible tag to see if the client should be able to see it or not.


```
typedef struct {
    Card cards[52];
    int top;
    int count;
} Deck;
```

The Deck data structure represents the deck of physical cards in a game. It holds Card cards, with DECK_SIZE being 52 representing the number of cards and int top which is an integer index holding the top of the deck. This data structure will be used primarily to keep track of the current state of the cards during a match.

AbilityCard:

```
typedef enum {
    NONE_ABILITY, SNIFF, ANT_TRAIL, POSE, LONG_TONGUE, WILD_GRAB
} AbilityType;
```

The AbilityType data structure represents all special abilities in our Anteater poker game.

```
typedef struct {
    AbilityType type;
    int used;
    int owner_seat;
    int target_seat;
    int param;
} AbilityCard;
```

The AbilityCard data structure represents all special details on any abilities in our Anteater poker game. Including when the card was put into play, who played the card, who the card targeted if anyone, and any other parameters that could be added on.

4.2. Detailed description of functions and parameters

Card `create_card`(Rank rank, Suit suit)

Instantiates and returns a new card structure with the specific rank and suit being initialized by the parameters.

int `is_valid_card`(Card card)

Validates whether or not the card is a legal card that can be used in poker.

int `cards_equal`(Card a, Card b)

Compare 2 card structures to check if they have the same suit and rank.

const char *`suit_to_string`(Suit suit)

Maps a numeric suit enumeration constant to its corresponding human-readable string literal.

const char *`rank_to_string`(Rank rank)

Maps a numeric rank enumeration constant to its corresponding human-readable string literal.

void `card_to_string`(Card card, char *buffer, int buffer_size)

Formats a card's rank and suit into a complete human-readable sentence and copies it into an allocated text buffer. It explicitly handles security boundary checks to prevent memory corruption.

void `print_card`(Card card)

A utility function that wraps `card_to_string` to easily print a human-readable representation of a card directly to standard output.

```
void init_deck(Deck *deck)
```

Initializes a deck structure by populating its internal array with a complete, standard 52-card deck ordered by suit and rank. It resets the tracking index to the beginning of the deck.

```
void shuffle_deck(Deck *deck)
```

Randomizes the sequence of the cards inside the deck array using the Fisher-Yates shuffle algorithm.

```
Card deal_card(Deck *deck)
```

Simulates dealing the next available card from the deck. It retrieves the card indexed by the current top pointer and then shifts the top pointer forward. Catches an error if no cards are left

```
int cards_remaining(const Deck *deck)
```

Returns the number of undealt cards based on the position of the tracking index.

```
int is_deck_empty(const Deck *deck)
```

Checks whether or not all the cards in the deck have been dealt, returns 1 if no cards are left.

```
void print_deck(const Deck *deck)
```

Iterates through the entire deck array and prints each card to the console. It marks already dealt cards and summarizes the total number of active cards left.

4.3. Detailed description of the communication protocol

```
int init_server(int port);
```

Initializes server-side listening socket. Creates a stream socket and binds it to the designated port passed through in the parameters. It then sets the socket to listening state.

```
int accept_client(int sock);
```

Called when the system select() flags an incoming connection request on the master socket. Runs system accept() to create a new communication channel for a new player.

```
void broadcast_game(int fds[], int num_clients, char *msg);
```

Iterates through all connected player channels to push real-time game state variables down the wire, forcing client screens to synchronize simultaneously.

```
NetMessage parse_client_msg(char *buf);
```

Deserializes incoming network packet streams into an organized application-layer structure that the state machine can read.

The communication protocol for Anteater Poker uses data packets to pass player actions and state updates between the server and client. Every data packet transmitted over the socket interface uses the layout [COMMAND_FLAG]:[PLAYER_ID]:[PAYLOAD_DATA], where the command flag is a 4-letter command string. When a client initiates an action, it sends standard flags like CONN to register a new session during an accept_client routine, or flags such as FOLD, CALL, or RAISE to transmit game-play decisions across the network. The server processes these incoming streams using parse_client_message and broadcasts state synchronization packets back to the table using flags like STAT to distribute current hand data, or WINS following a calc_winner() evaluation to distribute the pot chips.

5. Development plan and timeline

5.1. Partitioning of tasks

The project will be divided into tasks based on the major modules. One task group will focus on poker rules, including Texas Hold'em logic, shuffling, dealing, betting and the Anteater action card. Another task group will focus on the poker server application, which includes game state, turn coordination, point tracking and game ending. A separate task group will be on the graphical user interface, which will set up the users cards as well as displaying other players, action buttons, etc. Finally, the bot will be a task group focused on creating an automated player that can make logical and legal game decisions. This task partitioning allows all team members to contribute equally and efficiently.

5.2. Team member responsibilities

Jim Truong:

- Implementation of user interaction
 - Betting amount
 - Folding
 - Using action cards
 - Join/exit table

Ben Choi:

- Poker bot, Card/Deck implementation, Server creation

Shogo Stuck:

- Implementation of Rules module, game logic, AbilityCard

Hamza Vaid:

- Card/Deck implementation, Server and client creation, Rules module

Giovanna Dunker Estruquel:

- GTK client GUI including table display, players cards, action buttons, status messages

Back Matter

Copyright

© 2026 Anteater Poker Team (ZotHouse). All rights reserved.

This software was developed as part of the EECS 22L course at the University of California, Irvine. It is intended for educational use only. Anteater Poker uses points only and must not be used for gambling.

Error Messages

Error Message	Meaning
Cannot connect to server	The client cannot reach the host or port.
Name already in use	Another connected player is using the same display name.
Seat unavailable	The selected seat is occupied or locked.
Not your turn	The client attempts an action while another player is active.
Invalid raise amount	Raise is below minimum, above available points, or not a number.
Illegal action	The requested action is not legal in the current state.
Server disconnected	The network connection closed unexpectedly.
Deck error	The server detected an invalid or exhausted deck state.

References

- C Socket Documentation - <https://www.geeksforgeeks.org/c/socket-programming-cc/>
- GTK Documentation - <https://www.gtk.org/docs/>
- EECS 22L Project 2: Anteater Poker Handout, Prof. Rainer Dömer, May 4, 2026
- Anteater Poker User Manual, Version 0.1 Alpha Draft, ZotHouse Team, 2026.

Index

A

AbilityCard, 3, 6, 17
AbilityType, 17
Accept Client, 12, 20
Action Loop, 13
ActionType, 3, 6, 10
All-In, 3
Ante, 3
Anteater Ability Card, 3, 6, 9, 17
Automated Client, 9

B

Bet, 3
Big Blind, 3
Bluff, 3
Board, 3
Bot Player, 3, 7, 9, 11, 21
Broadcast, 3, 12, 20
Building, 15
Burn Card, 3

C

Call, 3, 9, 20
Card, 3, 6, 10, 16, 18
Card Images, 7
Card Structure, 16
Cards Equal, 18
Cards Remaining, 19
Check, 3, 9
Client, 3, 6, 7, 8, 9
Client Application, 6, 7, 8, 9
Client GUI, 7, 21
Client Module Hierarchy, 7
ClientState, 3, 6, 7, 8
Client-Server Communication, 6, 8, 12, 20
Command Flag, 20
Communication Protocol, 20
Community Cards, 3, 4, 10, 13
Compilation, 15
Configuration Options, 4, 14
Connect to Server, 8, 9

D

Data Structures, 6, 10, 16
Deck, 4, 10, 11, 16, 17, 19
Deck Error, 22
Deal Card, 19
Development Plan, 21
Download, 14

E

Error Messages, 22

F

Fisher-Yates Shuffle, 19
Flop, 4, 13
Fold, 4, 9, 20
Functions and Parameters, 18

G

Game Flow, 8
Game Logic, 6, 11, 21
Game State, 4, 6, 9, 10, 11, 13, 20
GameState, 4, 10
Glossary, 3
GTK, 4, 6, 7, 8, 9, 14, 21
GUI, 4, 6, 7, 8, 9, 11, 21

H

Hand Rankings, 5
Hole Cards, 4
Host, 14

I

Init Deck, 19
Init GUI, 8
Init Server, 12, 20
Installation, 14, 15
Invalid Raise Amount, 22
Is Deck Empty, 19
Is Valid Card, 18

L

Legal Action, 6, 9, 11
Lobby Phase, 13
Log File, 14

M

Main Pot, 4
Make Action, 8
Makefile, 15
Module Interfaces, 4, 8, 12
Muck, 4

N

Network Communication, 4, 6, 8, 20
NetworkMessage, 4, 6, 10, 20

P

Parse Client Message, 12, 20
Payload Data, 20
Player, 4, 10, 21
Player Actions, 3, 6, 8, 9, 20
PlayerAction, 4, 6, 9
PlayerView, 4, 6
Poker Bot, 9

Poker Client, 6
Poker Rules, 11, 21
Poker Server, 10
Port, 14
Pot, 4, 10, 11, 13
Pregame, 13
Private Cards, 4, 6, 9, 10
PrivateHand, 4, 6
Program Control Flow, 9, 13

R

Raise, 4, 8, 9, 20
Rank, 6, 16, 18
Rank to String, 18
References, 22
River, 4, 13
RuleResult, 5, 10
Rules Module, 11, 21

S

Scoreboard, 11, 13
Seat, 6, 13
Seat Unavailable, 22
Server, 5, 10, 11, 12, 13
Server Application, 10, 11, 12, 13
Server Control Flow, 13
Server GUI, 11
Server Module Hierarchy, 11
Showdown, 5, 13
Shuffle Deck, 19
Side Pot, 5
Small Blind, 5, 13
Socket, 5, 6, 8, 10, 12, 20
Socket Client, 7
Socket Server, 11
Suit, 6, 16, 18
Suit to String, 18
System Requirements, 14

T

Table of Contents, 2
TCP/IP, 5, 20
Team Member Responsibilities, 21
Turn, 5, 13

U

Unpacking, 14
Update GUI, 8
Use Ability, 6, 9

W

Winner Calculation, 12, 20
Wild Grab, 17